

Patrones de Software Utilizados

E-Commerce Zero · Trabajo Práctico Integrador N°1 · Ingeniería del Software II · 2026

Este documento resume cinco de los patrones aplicados en el sistema: **Capas**, **Modelo-Vista-Controlador (MVC)**, **DAO**, **OID** y **DTO**. Para cada uno se explica qué es, dónde se usa en el proyecto y se muestra un fragmento del código real. Los ejemplos usan el ABM de Nacionalidad, que es el CRUD de referencia del equipo, y el reporte de ventas.

Patrón	Dónde se aplica	Clases involucradas
Capas	Toda la aplicación	controllers, services, repositories, entities
MVC	Cada pantalla del sitio y del panel	38 controllers, 82 templates Thymeleaf, entidades y DTO
DAO	Acceso a la base de datos y a archivos	29 repositorios Spring Data JPA y NewsletterEnvioRepository
OID	Identificación de todas las entidades	BaseEntity (id UUID) y sus 29 entidades persistentes
DTO	Reportes, dashboard, catálogo y formularios	42 clases del paquete dto

1. Capas

Qué es

Organiza el software en niveles con una responsabilidad cada uno. Cada capa solo usa a la capa inmediatamente inferior, lo que permite cambiar una sin afectar a las demás.

Cómo se aplica

El sistema es web, con arquitectura Cliente-Servidor, y el servidor se divide en tres capas:

Capa	Paquete	Responsabilidad
Presentación	controllers + templates	Recibe las peticiones HTTP, llama al servicio y elige la vista. No contiene reglas de negocio.
Negocio	services	Validaciones (método validar), reglas del negocio y transacciones. No conoce la vista.
Persistencia	repositories	Lee y guarda las entidades en la base SQLite.

La regla del equipo es que **un controller nunca usa un repository**: siempre pasa por un service. Se puede verificar en el código: ningún controller importa una clase del paquete *repositories*.

```
// Presentación: el controller delega en el servicio
service.crearNacionalidad(nombre);

// Negocio: el servicio valida y usa el repositorio
public void crearNacionalidad(String nombre) throws ErrorServiceException {
    validar(nombre);
    Nacionalidad nacionalidad = new Nacionalidad();
    nacionalidad.setNombre(nombre.strip());
    repository.save(nacionalidad); // Persistencia
}
```

2. Modelo - Vista - Controlador (MVC)

Qué es

Separa la interfaz en tres partes: el **Modelo** (los datos y la lógica), la **Vista** (lo que ve el usuario) y el **Controlador** (recibe la acción del usuario, actualiza el modelo y elige la vista).

Cómo se aplica

- **Modelo:** las entidades (*entities*), los DTO y los servicios que los procesan.
- **Vista:** los templates Thymeleaf de *resources/templates*, que decoran los layouts *layout/publico.html* y *layout/admin.html*.
- **Controlador:** las clases *@Controller* de *controllers*, separadas en *publico*, *cliente* y *admin* según la sección del sitio.

Ejemplo con el listado de nacionalidades. El controlador pide los datos al servicio, los agrega al modelo y devuelve el nombre de la vista:

```
@Controller
@RequestMapping("/admin/nacionalidades")
public class NacionalidadController {

    @GetMapping
    public String listar(Model model) {
        model.addAttribute("pageTitle", "Nacionalidades");
        model.addAttribute("nacionalidades", service.listarNacionalidadActiva());
        return "admin/nacionalidades/listado"; // vista
    }
}
```

La vista *admin/nacionalidades/listado.html* solo muestra los datos que recibe en el modelo:

```
<html layout:decorate="~{layout/admin}">
<div layout:fragment="content">
    <a th:href="@{/admin/nacionalidades/nueva}">Nueva nacionalidad</a>
    <div th:replace="~{fragments/tabla :: tablaNombre(${nacionalidades},
                                                '/admin/nacionalidades')}"></div>
</div>
</html>
```

3. DAO (Data Access Object)

Qué es

Encapsula el acceso a los datos en un objeto propio. El resto del sistema pide o guarda objetos sin saber cómo ni dónde se almacenan (SQL, ORM o archivos).

Cómo se aplica

- **Con ORM:** 29 repositorios son interfaces que extienden *JpaRepository* de Spring Data JPA. Hibernate genera el SQL a partir del nombre de cada método.
- **Con archivos:** *NewsletterEnvioRepository* guarda la fecha del último envío del newsletter en el archivo *data/newsletter-ultimo-envio.txt*.

```
public interface NacionalidadRepository extends JpaRepository<Nacionalidad, String> {
    List<Nacionalidad> findAllByOrderByNombreAsc();
    List<Nacionalidad> findByEliminadoFalseOrderByNombreAsc();
    Optional<Nacionalidad> findByIdAndEliminadoFalse(String id);
}

@Repository
public class NewsletterEnvioRepository { // DAO sobre un archivo de texto
    public Optional<LocalDate> buscarUltimoEnvio() { ... }
    public void guardarUltimoEnvio(LocalDate fecha) throws ErrorServiceException { ... }
}
```

Los servicios reciben el DAO por inyección de dependencias en el constructor y nunca escriben SQL.

4. OID (Object Identifier)

Qué es

Cada objeto persistente tiene un identificador único que no depende de sus datos de negocio (nombre, CUIT, código). Con ese identificador se lo busca, se lo relaciona con otros objetos y se lo referencia desde la interfaz.

Cómo se aplica

Todas las entidades heredan de *BaseEntity*, que define el OID como un **UUID** generado por Hibernate. Si el nombre de una nacionalidad o el código de un producto cambian, su identificador sigue siendo el mismo.

```
@MappedSuperclass
public abstract class BaseEntity {

    @Id
    @UuidGenerator
    @Column(length = 36, nullable = false, updatable = false)
    private String id;

    @Column(nullable = false)
    private boolean eliminado = false;        // baja lógica
}
```

El OID se usa en todo el sistema:

- **Base de datos:** es la clave primaria de cada tabla y las claves foráneas apuntan a él.
- **DAO:** los repositorios se declaran con el tipo del OID, por ejemplo *JpaRepository<Nacionalidad, String>*, y se busca con *findByIdAndEliminadoFalse(id)*.
- **Vista y controladores:** las URLs identifican al objeto por su OID, por ejemplo */admin/nacionalidades/{id}/editar*.
- **Atributo *updatable* = false:** una vez asignado, el identificador no se puede modificar.

```
@GetMapping("/{id}/editar")
public String editar(@PathVariable String id, Model model) {
    Nacionalidad nacionalidad = buscarO404(id);        // busca por OID
    return formulario(model, id, nacionalidad.getNombre());
}
```

5. DTO (Data Transfer Object)

Qué es

Es un objeto simple, sin lógica, que solo transporta datos entre capas. Se usa cuando la vista necesita información armada a partir de varias entidades, como ocurre en los reportes.

Cómo se aplica

El paquete *dto* tiene 42 clases, la mayoría como *record* de Java (inmutables). Se usan en los reportes de ventas, stock y proveedores, en el dashboard, en el catálogo público y en los formularios. Ejemplo con el **reporte de ventas**:

```
public record ReporteVentasDTO(  
    int cantidadCompras,  
    int unidadesVendidas,  
    double montoTotal,  
    List<DetalleVentaDTO> detalle,  
    List<SubtotalFormaPagoDTO> subtotalesPorFormaPago) {}  
  
public record DetalleVentaDTO(LocalDate fechaCompra, String producto, String categoria,  
    int cantidad, double precioUnitario, double subtotal,  
    String identificadorCompra, String formaPago, String idFactura, String idOrden) {}
```

El servicio recorre las facturas pagadas del período, arma el DTO y lo devuelve. El controlador lo pasa a la vista sin exponer las entidades:

```
// ReporteVentasService (negocio)  
public ReporteVentasDTO generar(LocalDate desde, LocalDate hasta) throws ErrorServiceException {  
    ...  
    return new ReporteVentasDTO(cantidadCompras, unidadesVendidas, montoTotal,  
        List.copyOf(detalle), subtotales);  
}  
  
// ReporteVentasController (presentación)  
model.addAttribute("reporte", service.generar(desdeEfectivo, hastaEfectivo));  
return "admin/reportes/ventas";
```

La vista usa directamente los campos del DTO, por ejemplo `${reporte.montoTotal}` y `${reporte.detalle}`.